

Scope for E5

1. Convert an evaluated thesis into a concrete TradePlan.
 - TradePlan expresses what to trade (per asset legs, direction, notional) given a chosen capital amount.
 - **E5 v1 uses a simple, pluggable sizing policy (“NaiveSizingEngine”) that mirrors thesis expression legs into TradeLeg.size_pct weights given a total_notional. Future iterations may replace this with a risk-aware sizing engine without changing public DTOs or endpoints.**
2. Execute that TradePlan in a paper portfolio.
 - Create Trade records via TradeRepository.
 - Fill at current market prices (no slippage, no partial fills).
 - Update portfolio state so DataAccess.get_current_portfolio() reflects new positions.
3. Expose an approve endpoint.
 - POST /api/v1/thesis/{thesis_id}/approve-plan:
 - Validates the thesis is known and not already active.
 - Builds a TradePlan from thesis + notional (using the same sizing policy as E5 v1: NaiveSizingEngine).
 - Executes the plan.
 - Marks thesis as ACTIVE.
4. Basic P&L / valuation.
 - Continued use of DataAccess.get_current_portfolio() and get_portfolio_depth() for valuation.
 - Add a clean, deterministic way to compute P&L per thesis for UI (simple helper).
5. Testing:
 - Unit tests for Plan creation and execution using stubbed PriceSource.
 - Integration test for approve endpoint: trades appear in repo and portfolio updates.

Explicit constraint to keep complexity under control:

- **E5 v1 is long-only.**

Theses with short legs cannot be paper-executed; approval endpoint returns 400 for those.

Short support + correct cash/margin accounting is a separate iteration.

That removes the nasty cash / net-equity edge cases.

E5 Design

1. Models

1.1 TradeLeg / TradePlan

```
class TradeLeg(BaseModel):
    asset: str
    direction: Literal["LONG"] # v1: long-only
    size_pct: float             # percent of total notional (0-100)

class TradePlan(BaseModel):
    thesis_id: str
    total_notional: float       # dollars to deploy into this thesis
    legs: list[TradeLeg]        # weights that sum <= 100
```

Notes:

- **E5 v1 sizing policy (NaiveSizingEngine):** size_pct mirrors thesis expression legs directly. We enforce $\text{sum}(\text{size_pct}) \leq 100$.
- Any unallocated percentage stays in cash (we do not model cash explicitly; it just means not all capital is deployed).
- **Later phases may introduce a more sophisticated sizing engine (e.g. risk-aware, vol-targeting) that still emits the same TradePlan shape; only the logic that chooses size_pct values changes.**

1.2 SizingConstraints / SizingEngine (sizing abstraction)

We introduce a conceptual sizing abstraction so position logic is not permanently welded into PaperExecutionAdapter. E5 v1 *inlines* a naive implementation of this interface inside `create_plan_from_thesis`, but the API is defined now to keep a clean upgrade path.

```
class SizingConstraints(BaseModel):
    """
    Configuration for how much capital/risk a thesis is allowed to take.
    E5 v1 uses simple, static defaults; later phases can make this dynamic
    and
    risk-aware without changing the TradePlan DTO.
    """
    max_gross_leverage: float = 2.0
```

```

    max_position_weight: float = 0.15    # fraction of total_notional
    max_risk_per_thesis: float = 0.02    # fraction of equity (interpreted by
engine)

```

```

class SizingEngine(Protocol):
    """
    Pluggable policy for turning a thesis + total_notional (+ optional
constraints)
    into a TradePlan.

    E5 v1 behavior (NaiveSizingEngine) is implemented directly in
PaperExecutionAdapter.create_plan_from_thesis for simplicity, but that
method can later be refactored to delegate to an injected SizingEngine
without changing its signature or the TradePlan schema.
    """
    def size_plan(
        self,
        thesis: Thesis,
        total_notional: float,
        constraints: SizingConstraints | None = None,
        # future: portfolio_snapshot: RiskSnapshot | None = None,
    ) -> TradePlan:
        ...

```

NaiveSizingEngine (E5 v1 semantics):

- Mirrors thesis.expression_legs 1:1 into TradeLegs:
 - all legs LONG,
 - size_pct = leg.size_pct,
 - $\text{sum}(\text{size_pct}) \leq 100$.
- Ignores max_risk_per_thesis and portfolio-level information.
- Enforces only:
 - total_notional > 0,
 - no SHORT legs,
 - allocations sum ≤ 100 .

Later, a RiskAwareSizingEngine can consume risk reports, volatility, and constraints while returning the same TradePlan shape.

1.3 P&L helper DTO (for per-thesis performance)

```

class ThesisPnL(BaseModel):
    thesis_id: str
    invested_notional: float
    current_value: float
    unrealized_pnl: float
    unrealized_pnl_pct: float | None

```

This is an internal DTO returned by a DataAccess helper for UI consumption.

2. PaperExecutionAdapter

File: src/slice/execution/paper.py

```
class PaperExecutionAdapter:
    """
    Paper-execution harness for turning TradePlans into Trade rows.

    Conceptually depends on a SizingEngine to build TradePlans from Theses.
    E5 v1 inlines a NaiveSizingEngine implementation inside
    create_plan_from_thesis; later phases may inject a different sizing
engine
    while preserving this class's public methods and the TradePlan DTO.
    """

    def __init__(self, trade_repo: TradeRepository, price_source:
PriceSource):
        self.trade_repo = trade_repo
        self.price_source = price_source
        # future: self.sizing_engine: SizingEngine = NaiveSizingEngine(...)
    def create_plan_from_thesis(
        self,
        thesis: Thesis,
        total_notional: float,
    ) -> TradePlan:
        """
        Use the current sizing policy (E5 v1: NaiveSizingEngine) to convert
thesis.expression_legs (long-only) into a TradePlan for the given
total_notional.

        E5 v1 policy:
        - total_notional must be positive
        - all legs must be LONG
        - allocations sum <= 100 (unallocated capital implicitly stays in
cash)

        Later phases may delegate this to an injected SizingEngine that uses
portfolio/risk information, without changing this method's signature
or the TradePlan DTO.
        """
        if total_notional <= 0:
            raise ValueError("total_notional must be positive")

        legs: list[TradeLeg] = []
        total_pct = 0.0

        for leg in thesis.expression_legs:
            if leg.direction.upper() != "LONG":
```

```

        raise ValueError("PaperExecutionAdapter v1 only supports long
legs")

        legs.append(
            TradeLeg(
                asset=leg.asset,
                direction="LONG",
                size_pct=leg.size_pct,
            )
        )
        total_pct += leg.size_pct

    if total_pct > 100.0 + 1e-6:
        raise ValueError(f"Expression legs sum to {total_pct}%, must be
<= 100")

    return TradePlan(
        thesis_id=thesis.id,
        total_notional=total_notional,
        legs=legs,
    )

def execute_plan(
    self,
    plan: TradePlan,
    as_of: date | None = None,
) -> list[Trade]:
    """
    Execute plan legs as paper trades at current prices.
    Long-only, immediate fill, no costs.

    This method is intentionally dumb in E5 v1: no slippage, no fees,
    no leverage/margin checks. It assumes the sizing policy has already
    produced an acceptable TradePlan.
    """
    if not plan.legs:
        return []

    exec_dt = (
        datetime.combine(as_of, datetime.min.time())
        if as_of is not None
        else datetime.utcnow()
    )

    trades: list[Trade] = []

    for leg in plan.legs:
        notional = plan.total_notional * (leg.size_pct / 100.0)
        if notional <= 0:
            continue

        price = self.price_source.get_current_price(leg.asset)
        if price <= 0:
            raise RuntimeError(f"Invalid price {price} for asset
{leg.asset}")

        quantity = notional / price

```

```

trade = Trade(
    trade_id=str(uuid4()),
    timestamp=exec_dt,
    asset=leg.asset,
    action="BUY",    # long-only, v1
    quantity=quantity,
    price=price,
    type="PAPER",
    thesis_ref=plan.thesis_id,
)
self.trade_repo.insert(trade)
trades.append(trade)

return trades

```

- **No shorts.**
- **No cash or margin modeling.**
- TradeRepo is the single source of truth; DataAccess.get_current_portfolio() already aggregates trades.

3. P&L and Thesis-Level Performance

We add a deterministic helper in DataAccess to compute per-thesis performance from trades.

```

class DataAccess:
    # existing fields...

    def get_thesis_pnl(self, thesis_id: str) -> ThesisPnL:
        trades = self.trade_repo.list_by_thesis(thesis_id)
        if not trades:
            return ThesisPnL(
                thesis_id=thesis_id,
                invested_notional=0.0,
                current_value=0.0,
                unrealized_pnl=0.0,
                unrealized_pnl_pct=None,
            )

        # Long-only assumption: all actions are BUY
        invested_notional = sum(t.quantity * t.price for t in trades)

        # Current value at latest prices
        # Group by asset to reduce price calls
        by_asset: dict[str, float] = {}
        for t in trades:
            by_asset[t.asset] = by_asset.get(t.asset, 0.0) + t.quantity

        current_value = 0.0
        for asset, qty in by_asset.items():

```

```

        price = self.price_source.get_current_price(asset)
        current_value += qty * price

    unrealized_pnl = current_value - invested_notional
    unrealized_pnl_pct = None
    if invested_notional > 0:
        unrealized_pnl_pct = unrealized_pnl / invested_notional * 100.0

    return ThesisPnL(
        thesis_id=thesis_id,
        invested_notional=invested_notional,
        current_value=current_value,
        unrealized_pnl=unrealized_pnl,
        unrealized_pnl_pct=unrealized_pnl_pct,
    )

```

UI can call this (via a lightweight API wrapper) to populate the “P&L” column in a thesis list.

No attempt to model shorts or leverage; the math is exactly “what did we pay vs what is it worth now”.

4. Integrating E5 with E4

4.1 Evaluation session output

In E4 we already allowed an optional `trade_plan` on `ThesisEvaluationSessionResult`. We now make that explicit:

- The evaluation session will optionally include a default plan (v1), using a fixed notional **and the same sizing policy as E5 v1 (NaiveSizingEngine behavior encoded in `create_plan_from_thesis`)**.

```

DEFAULT_EVAL_NOTIONAL = 100_000.0 # or config

# in ThesisEvaluationSession.run(...)
trade_plan = None
if self.exec_adapter is not None:
    try:
        trade_plan = self.exec_adapter.create_plan_from_thesis(
            thesis=thesis,
            total_notional=DEFAULT_EVAL_NOTIONAL,
        )
    except ValueError:
        # e.g. shorts; in that case evaluation still succeeds, but no plan
        trade_plan = None

```

That plan is purely illustrative – it shows the shape of the expression. Approval will recompute a plan with a notional chosen by the user or default.

Later, when a RiskAwareSizingEngine exists, E4 remains agnostic: it only consumes the resulting TradePlan, regardless of how it was sized internally.

4.2 Approve Plan API

Route: POST /api/v1/thesis/{thesis_id}/approve-plan

Request:

```
class ApprovePlanRequest(BaseModel):
    total_notional: float | None = None # optional override
```

Implementation:

```
@router.post("/api/v1/thesis/{thesis_id}/approve-plan")
def approve_plan(
    thesis_id: str,
    body: ApprovePlanRequest | None = None,
    data_access: DataAccess = Depends(get_data_access),
    price_source: PriceSource = Depends(get_price_source),
):
    thesis = data_access.thesis_repo.get_by_id(thesis_id)
    if thesis is None:
        raise HTTPException(status_code=404, detail="Thesis not found")

    # Prevent double-approval
    if getattr(thesis, "status", None) == "ACTIVE":
        raise HTTPException(status_code=400, detail="Thesis already active")

    adapter = PaperExecutionAdapter(
        trade_repo=data_access.trade_repo,
        price_source=price_source,
    )

    notional = (
        body.total_notional
        if body is not None and body.total_notional
        else 100_000.0
    )

    try:
        plan = adapter.create_plan_from_thesis(
            thesis,
            total_notional=notional,
        )
```



```

except ValueError as e:
    # shorts, bad allocations, or non-positive notional
    raise HTTPException(status_code=400, detail=str(e))

try:
    trades = adapter.execute_plan(plan)
except Exception as e:
    # execution is atomic enough in this context; if partial inserts ever
    # happen, they show in DB - we accept that risk at this stage
    raise HTTPException(status_code=500, detail=f"Execution failed: {e}")

# Mark thesis ACTIVE
thesis.status = "ACTIVE"
data_access.thesis_repo.update(thesis)

return {
    "status": "success",
    "thesis_id": thesis_id,
    "total_notional": notional,
    "executed_trades": [t.dict() for t in trades],
}

```

- No reuse of E4's plan object. That's intentional: approval is allowed to choose its own notional.
- If the thesis contains any short legs, `create_plan_from_thesis` fails → 400; user cannot paper-trade that thesis in v1.
- **When a different sizing engine is introduced later, this endpoint still calls `create_plan_from_thesis` and remains unchanged; only the internal sizing policy changes.**

5. Testing Plan

5.1 Unit tests – plan generation

These tests document the **NaiveSizingEngine** behavior in E5 v1 (1:1 mapping from thesis expression legs to `TradeLeg.size_pct`). Future engines may change sizing behavior, but must either preserve these semantics or update the tests and doc explicitly while keeping the `TradePlan` schema and `create_plan_from_thesis` signature unchanged.

Scenarios:

1. Basic 60/40 long-only thesis.
 - Thesis expression legs: 60% A, 40% B, both LONG.
 - `total_notional=10_000`.

- `create_plan_from_thesis` returns:
 - `thesis_id` matches.
 - `total_notional=10_000`.
 - legs: 2 legs with the same assets and `size_pct` 60 and 40.
 - No exceptions.
- 2. Allocations sum < 100.
 - 50% A, 25% B; sum 75%.
 - Plan is accepted as-is (remaining 25% is cash/unused).
- 3. Allocations sum > 100 → error.
 - 70% A, 40% B (110 total).
 - `create_plan_from_thesis` raises `ValueError`.
- 4. Short leg → error.
 - Thesis has leg with `direction="SHORT"`.
 - `create_plan_from_thesis` raises `ValueError`.
- 5. Non-positive notional → error.
 - `total_notional <= 0` raises `ValueError`.

5.2 Unit tests – plan execution

Use a stub `PriceSource` returning fixed prices.

1. Two-leg execution, long-only.
 - Plan: 60% A, 40% B, `total_notional=10_000`.
 - Prices: A: 50, B: 200.
 - `execute_plan`:
 - Creates 2 trades.
 - Trade1: asset A, action BUY, quantity $\approx (6000 / 50) = 120$.
 - Trade2: asset B, action BUY, quantity $\approx (4000 / 200) = 20$.
 - Persists both via `trade_repo.insert`.
 - Timestamp set to `as_of` if provided, else `datetime.utcnow()`.
 - Check that `trade_repo.list_by_thesis(thesis_id)` returns those trades.
2. Zero-sized legs filtered.
 - Plan with a leg `size_pct=0`.
 - That leg should be skipped; no trade inserted.
3. Bad price → error.
 - `PriceSource` returns 0 or negative for some asset.
 - `execute_plan` raises `RuntimeError` – no trade for that asset should be inserted. (If you want stricter atomicity, wrap the repo in a transaction; for now, that's overkill.)

5.3 DataAccess P&L helper

1. Simple long-only scenario.
 - Trades: one BUY 100 A at \$50.
 - PriceSource current: A = 60.
 - get_thesis_pnl:
 - $\text{invested_notional} = 100 * 50 = 5000$.
 - $\text{current_value} = 100 * 60 = 6000$.
 - $\text{unrealized_pnl} = 1000$.
 - $\text{unrealized_pnl_pct} = 20.0$.
2. Multiple trades same asset.
 - BUY 50 at 50, BUY 50 at 60.
 - Price now 55.
 - $\text{invested_notional} = 50*50 + 50*60 = 5500$.
 - $\text{current_value} = 100*55 = 5500$.
 - P&L = 0, pct = 0.
3. No trades.
 - Returns zeros and $\text{unrealized_pnl_pct}=\text{None}$.

5.4 Approve-plan endpoint integration

Using FastAPI TestClient + dependency overrides (test repos, test price source):

1. Happy path.
 - Insert a thesis with 2 long legs.
 - PriceSource stub returns positive prices.
 - POST /api/v1/thesis/{id}/approve-plan with { "total_notional": 50_000 }.
 - Assert:
 - 200 response, "status": "success".
 - Response lists 2 executed trades.
 - `trade_repo.list_by_thesis` returns those trades.
 - `thesis.status` is now ACTIVE.
 - `DataAccess.get_current_portfolio()` sees non-empty positions.
2. Double approval blocked.
 - Call approve once → 200.
 - Call approve again → 400 with “already active” message.
 - Trade count in repo remains unchanged between calls.
3. Short thesis not allowed.
 - Thesis with a short leg.
 - Approve plan → 400 with a clear error message from `create_plan_from_thesis`.
4. Missing thesis.
 - Unknown thesis_id.
 - Approve plan → 404.
5. Execution error path (e.g. bad price).
 - Stub PriceSource.get_current_price to raise for some asset.

- Approve plan → 500; confirm no extra trades for that thesis are visible (depending on how you implement error path; if you want full atomicity, use DB transaction in implementation and assert zero trades).
-

E5 – Risks & Explicit Non-Features

- **No shorts in v1.**

This is deliberate. Proper short support requires explicit cash/margin modeling and net-equity semantics that aren't worth half-doing. E5 explicitly rejects theses with short legs.

- **No transaction-cost or slippage modeling.**

- All trades fill at the stub “current price”.
- This is fine for a test harness; if you later care, you introduce slippage/fees in the adapter.

- **No cash position tracking.**

- We treat `invested_notional` as sum of BUY trade costs.
- `P&L` = current value minus `invested_notional`. No explicit cash bookkeeping or leverage constraints.

- **Plan mismatch between E4 and E5.**

- Evaluation session's plan (E4) is default/illustrative.
- Approve endpoint recomputes plan from thesis and provided notional.
- That is acceptable for this harness and simpler than caching plans server-side.

- **Partial execution atomicity.**

- With the current code shape, if a mid-loop insert fails, you could theoretically get a partial set of trades.
- At this stage, you accept that risk or wrap the insert loop in a DB transaction when you implement it.

- **No risk-aware sizing or portfolio-integrated risk budgets in E5 v1.**

- The `NaiveSizingEngine` simply mirrors thesis expression legs into `TradeLeg.size_pct` and uses `total_notional` as a scalar book size.
- It does **not** look at portfolio volatility, VaR, factor exposures, or per-thesis risk budgets.
- A later phase is expected to introduce a `RiskAwareSizingEngine` that consumes the existing risk/report machinery (VaR, concentration, factor exposures, etc.) while keeping:
 - `TradePlan` / `TradeLeg` schemas stable, and
 - `PaperExecutionAdapter.create_plan_from_thesis` and the approve endpoint signatures unchanged.

Net: this E5 gives you a usable, testable paper-trading path – Thesis → TradePlan → Trades → Portfolio/P&L – while explicitly treating sizing as a pluggable policy and **not** pretending to be a full prime broker or a complete risk-based sizing system.